چالش ۳۰ روزهٔ اندروید

آمادگی مصاحبهٔ اندروید — جونیور ← مید ← سینیور ← آرکیتکت

نسخهٔ ۲۰۲۶

روز ۸

مبانی Jetpack Compose — state، recomposition، side effectها و stability



اسماعیل مرادی

@esmaeeil-moradi700

چرا Compose روز هشتم است؟

Compose ابزار پیش‌فرض UI برای صفحه‌های جدید اندروید است و سؤال‌های مصاحبه از «UI اعلانی چیست؟» عبور کرده‌اند. حالا می‌پرسند **Compose چطور تصمیم می‌گیرد چه چیزی را دوباره اجرا کند:**

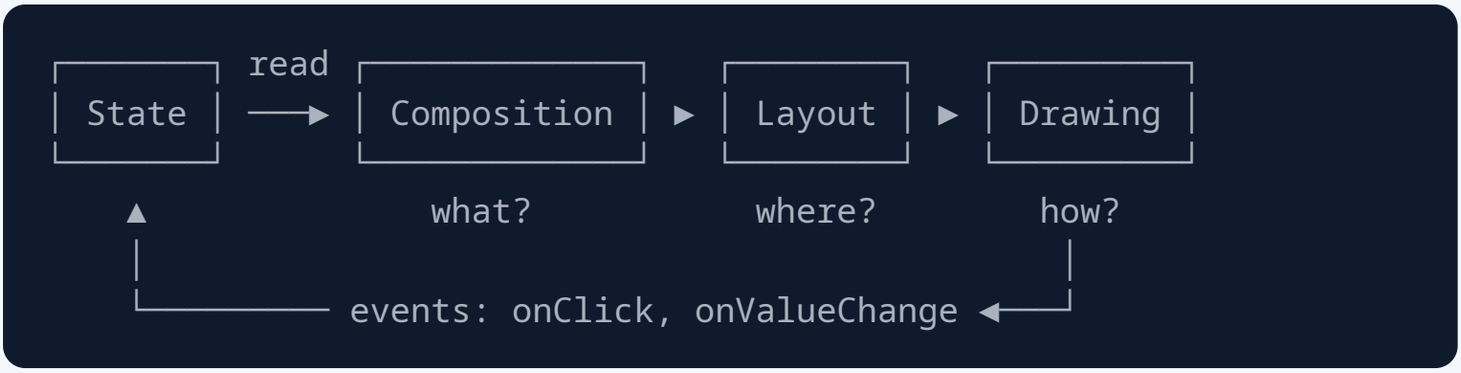
- recomposition، skip شدن و اینکه state را **کجا** می‌خوانید
- `remember` و `rememberSaveable` و اینکه هر state از چه چیزی جان سالم به در می‌برد
- APIهای side effect — و باگ‌هایی که با استفادهٔ غلط از آن‌ها می‌سازید
- stability و **strong skipping mode** که از Kotlin 2.0.20 به‌صورت پیش‌فرض روشن است
- تم، design system و همان UI روی iOS با Compose Multiplatform

سطح	سؤال‌ها
جونیور	recomposition، state و remember، بالا بردن state
مید	side effectها، modifierها، لیست‌های lazy و key، derivedStateOf
سینیور	stability و strong skipping، فازها و کارایی، CompositionLocal
آرکیتکت	تم Material 3، API کامپوننت‌های design system، Compose در مقیاس بزرگ



اسماعیل مرادی
@esmaeeil-moradi700

مدل ذهنی: UI تابعی از state است



- هر composable، UI را برای state فعلی **توصیف** می‌کند؛ viewها را تغییر نمی‌دهد
- نوشتن در یک state object فقط برای scopeهایی کار زمان‌بندی می‌کند که آن را **می‌خوانند**
- جایی که state را می‌خوانید تعیین می‌کند کدام فاز دوباره اجرا شود: composition، layout یا فقط drawing

سؤالی که پشت هر جواب COMPOSE است

«*چه کسی این state را می‌خواند، و در کدام فاز؟*» — باگ‌های recomposition، مشکلات کارایی و باگ‌های effect همه به همین سؤال برمی‌گردند.



اسماعیل مرادی
@esmaeeil-moradi700

جونیور

recomposition، state و remember، بالا بردن state



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱. composition و recomposition

```
@Composable
fun Greeting(name: String, modifier: Modifier = Modifier) {
    Text(text = "Hello, $name", modifier = modifier)
}

@Composable
fun Counter() {
    var count by remember { mutableIntStateOf(0) }
    Button(onClick = { count++ }) {
        Text("Clicked $count times")
    }
}
```

- **composition:** اجرای اول درخت UI را می‌سازد و مقدارهای rememberشده را در *slot table* نگه می‌دارد
- **recomposition:** وقتی `count` تغییر می‌کند، Compose فقط composableها و lambdaهایی را دوباره اجرا می‌کند که آن را **می‌خوانند**
- **skip شدن:** composableی که ورودی‌هایش تغییر نکرده اصلاً دوباره اجرا نمی‌شود



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱. چیزهایی که مصاحبه‌گر چک می‌کند

تلهٔ مصاحبه

«*recomposition کل صفحه را از نو می‌کشد.*» — **نه.** Compose کوچک‌ترین scopeهایی را که state تغییرکرده را می‌خوانند دوباره اجرا می‌کند و هر چیزی را که ورودی‌هایش برابر است skip می‌کند.

تلهٔ مصاحبه

«*composable یک بار، از بالا به پایین اجرا می‌شود.*» — ممکن است skip شود، در هر فریم انیمیشن دوباره اجرا شود، یا با هر ترتیبی اجرا شود؛ و یک recomposition ممکن است **لغو** و دور ریخته شود. پس بدنهٔ composable باید سریع، idempotent و بدون side effect باشد.

سؤال تکمیلی

«*چرا `remember` داخل حلقهٔ `for` وقتی آیتم‌ها جابه‌جا می‌شوند stateها را قاطی می‌کند؟*» — `remember` **موقعیتی** است: کلیدش جای آن در درخت فراخوانی است. هر آیتم را داخل `key(item.id) { … }` بگذارید تا state همراه خود آیتم جابه‌جا شود.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۲. state: چه چیزی از چه چیزی عبور می‌کند

API	عبور می‌کند از	کاربرد
`remember { mutableStateOf() }`	recomposition	state عنصر UI: باز بودن، انتخاب‌شدن
`rememberSaveable { … }`	علاوه بر آن، configuration change **و** process death	ورودی کوچک کاربر: متن، تب فعلی
`StateFlow` در ViewModel	علاوه بر recomposition، configuration change (**نه** process death)	state صفحه و منطق بیزینس
repository یا دیسک	همه چیز	خود داده

```
var query by rememberSaveable { mutableStateOf("") }
var expanded by remember { mutableStateOf(false) }
val selected = remember { mutableStateListOf<String>() }
val state by viewModel.uiState.collectAsStateWithLifecycle()
```



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۲. تله‌های state

تلهٔ مصاحبه

`val count = mutableStateOf(0)` **بدون** `remember` — در هر recomposition یک state object تازه ساخته می‌شود و مقدار مدام صفر می‌شود. Lint آن را با `UnrememberedMutableState` علامت می‌زند.

تلهٔ مصاحبه

`mutableStateOf(mutableListOf<Item>())` و بعد `.add(item)` — Compose فقط **نوشتن در خود state object** را می‌بیند، نه تغییر داخل یک لیست معمولی. یک لیست تغییرناپذیر جدید assign کنید، یا از `mutableStateListOf` استفاده کنید.

تلهٔ مصاحبه

گذاشتن یک لیست بزرگ در `rememberSaveable` — این داده در `Bundle` مربوط به saved state می‌رود که کوچک است و ممکن است `TransactionTooLargeException` بدهد. شناسه‌ها را ذخیره کنید و داده را دوباره بخوانید. تایپ‌های سفارشی به `@Parcelize` یا یک `Saver` نیاز دارند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۳. بالا بردن state: composableهای بدون state

```
@Composable
fun SearchBar(
    query: String,
    onQueryChange: (String) -> Unit,
    modifier: Modifier = Modifier,
) {
    OutlinedTextField(
        value = query,
        onValueChange = onQueryChange,
        modifier = modifier.fillMaxWidth(),
        singleLine = true,
    )
}
```

- state را تا **پایین‌ترین جدِّ مشترکی** بالا ببرید که آن را می‌خواند یا می‌نویسد: مقدار به پایین، lambdaی event به بالا
- composableهای بدون state به‌راحتی preview، تست و بازاستفاده می‌شوند
- پارامتر `modifier: Modifier = Modifier` **اولین پارامتر اختیاری** است و فقط یک بار روی ریشه اعمال می‌شود



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۳. state کجا باید زندگی کند

تلهٔ مصاحبه

پاس دادن `ViewModel` به composableهای فرزند. هر فرزند را به یک صفحهٔ خاص گره می‌زند و preview را خراب می‌کند. فرزندها **state** و **lambda** می‌گیرند؛ فقط route در سطح صفحه با ViewModel کار دارد (روز ۳).

تلهٔ مصاحبه

«*همه چیز را به ViewModel ببرید.*» — state خالص عنصر UI (موقعیت اسکرول، باز بودن یک کارت، فوکوس) می‌تواند در composition یا در یک **plain state holder** بماند. state بیزینسی جایش در ViewModel است.

سؤال تکمیلی

«*plain state holder چیست؟*» — کلاسی که منطق و state یک بخش UI را کنار هم نگه می‌دارد و با یک تابع `rememberXState()` ساخته می‌شود — مثل `LazyListState` یا `DrawerState`. تابع `rememberSearchState()` خودتان هم همین الگو را دنبال می‌کند.



اسماعیل مرادی
@esmaeeil-moradi700

میدِ

side effectها، modifierها، لیست‌های lazy و key، derivedStateOf



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۴. APIهای side effect

API	طول عمر	کاربرد
`LaunchedEffect(key)`	coroutine با ورود شروع می‌شود، با تغییر key از نو شروع می‌شود و با خروج لغو می‌شود	کار suspend وابسته به UI: نمایش snackbar، اجرای انیمیشن
`rememberCoroutineScope()`	وقتی فراخوانی از composition خارج شود، scope لغو می‌شود	اجرای کار از callbackها: اسکرول یا باز کردن drawer در `onClick`
`DisposableEffect(key)`	راه‌اندازی به‌علاوهٔ `onDispose` اجباری	ثبت و حذف listenerها
`SideEffect`	بعد از هر recomposition موفق	رساندن state کامپوز به کدی که Compose نیست
`rememberUpdatedState(x)`	آخرین مقدار، بدون شروع دوبارهٔ effect	effect طولانی که یک lambdaی متغیر را صدا می‌زند
`produceState`	coroutineی که یک `State` تولید می‌کند	تبدیل منبع callback یا suspend به `State`

تلهٔ مصاحبه

«با `rememberCoroutineScope` *باید خودتان coroutineها را لغو کنید.*» — **غلط است** (و نسخهٔ پارسال همین را گفته بود). وقتی composable از composition خارج شود، scope خودکار لغو می‌شود.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۴. DisposableEffect در عمل

```
@Composable
fun ScreenAnalytics(onStart: () -> Unit) {
    val lifecycleOwner = LocalLifecycleOwner.current
    val currentOnStart by rememberUpdatedState(onStart)

    DisposableEffect(lifecycleOwner) {
        val observer = LifecycleEventObserver { _, event ->
            if (event == Lifecycle.Event.ON_START) {
                currentOnStart()
            }
        }
        lifecycleOwner.lifecycle.addObserver(observer)
        onDispose {
            lifecycleOwner.lifecycle.removeObserver(observer)
        }
    }
}
```

- key همان **lifecycle owner** است: owner جدید یعنی dispose و راه‌اندازی دوباره
- `rememberUpdatedState` جدیدترین lambda را بدون شروع دوبارهٔ effect به آن می‌دهد
- میان‌بر در `lifecycle-runtime-compose`: یعنی `LifecycleStartEffect` و `LifecycleResumeEffect`

تلهٔ مصاحبه

بارگذاری داده در `LaunchedEffect(Unit)`. هر بار که صفحه دوباره وارد composition شود — مثلاً بعد از هر چرخش صفحه — دوباره اجرا می‌شود. بارگذاری جایش در ViewModel است (روز ۳).



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۵. modifierها: ترتیب یعنی معنا

```
Text(
    text = "Pay",
    modifier = Modifier
        .padding(16.dp)        // outer margin, not clickable
        .clip(RoundedCornerShape(12.dp))
        .background(MaterialTheme.colorScheme.primary)
        .clickable(onClick = onPay)
        .padding(horizontal = 24.dp, vertical = 12.dp),
)
```

- هر modifier همهٔ modifierهای بعد از خودش را **در بر می‌گیرد**؛ زنجیره از بیرون به داخل اعمال می‌شود
- ترتیب، اندازه، ناحیهٔ لمس، clip و رسم را عوض می‌کند: جای `clip` و `background` را عوض کنید و گوشه‌های گرد از بین می‌روند
- زنجیرهٔ modifier یک مقدار تغییرناپذیر است — ساختنش ارزان و بازاستفاده‌اش امن است

تلهٔ مصاحبه

نوشتن modifier سفارشی با `Modifier.composed { }`. به دلایل کارایی دیگر توصیه نمی‌شود — `Modifier.Node` را همراه با یک `ModifierNodeElement` پیاده کنید، یا modifierهای موجود را در یک extension function ساده زنجیر کنید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۶. لیست‌های lazy: key و content type

```
LazyColumn(state = listState) {
    items(
        items = messages,
        key = { it.id },
        contentType = { it.kind },
    ) { message ->
        MessageRow(message, Modifier.animateItem())
    }
}
```

- فقط آیتم‌های قابل‌مشاهده (و prefetchشده) compose می‌شوند
- پارامتر `key` : state آیتم و موقعیت اسکرول همراه **خود آیتم** می‌مانند، نه اندیسش — بدون آن، key همان موقعیت است
- پارامتر `contentType` : آیتم‌های هم‌نوع می‌توانند composition همدیگر را بازاستفاده کنند
- `Modifier.animateItem()` اضافه شدن، حذف و جابه‌جایی را انیمیت می‌کند — و به key نیاز دارد



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۶. تله‌های لیست lazy

تلهٔ مصاحبه

یک `LazyColumn` داخل `Column` ی که `verticalScroll` دارد crash می‌کند: لیست lazy را نمی‌شود با ارتفاع بی‌نهایت اندازه‌گیری کرد. از **یک** لیست lazy با چند بخش `item { }` و `items { }` استفاده کنید.

تلهٔ مصاحبه

keyها باید **یکتا** باشند (key تکراری crash می‌دهد) و در اندروید از نوعی باشند که `Bundle` بتواند ذخیره کند — مثل `String`، `Int` و `Long`. کل data object را key نکنید.

تلهٔ مصاحبه

آیتم‌هایی که قبل از رسیدن داده اندازهٔ صفر دارند (مثلاً تصویرهای خالی). لیست فکر می‌کند همه در صفحه جا می‌شوند و خیلی بیشتر از لازم compose می‌کند. به placeholderها اندازه بدهید.

سؤال تکمیلی

«*برای ۲۰ ردیف ثابت، Column یا LazyColumn؟*» — `Column` همهٔ فرزندها را از اول compose و layout می‌کند؛ برای محتوای کوچک و محدود مشکلی نیست. دادهٔ طولانی یا نامحدود یعنی lazy.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۷. derivedStateOf یا remember(key)؟

```
val listState = rememberLazyListState()

// Recomposes whenever the first visible index changes
val showBad = listState.firstVisibleItemIndex > 0

// Recomposes only when the Boolean flips
val showScrollToTop by remember {
    derivedStateOf { listState.firstVisibleItemIndex > 0 }
}
```

ابزار	ورودی‌ها	کِی دوباره حساب می‌شود
`remember(a, b) { f(a, b) }`	پارامترها یا مقدارهای معمولی	وقتی یکی از keyها تغییر کند
`derivedStateOf { }`	state objectهایی که **بیشتر** از نتیجه تغییر می‌کنند	وقتی خود نتیجه تغییر کند

تلهٔ مصاحبه

`remember { derivedStateOf { count > 0 } }` وقتی `count` یک **پارامتر** معمولی است، نه state — هیچ‌وقت ردیابی نمی‌شود و مقدار برای همیشه کهنه می‌ماند. از `remember(count) { … }` استفاده کنید یا مستقیم حسابش کنید.



اسماعیل مرادی
@esmaeeil-moradi700

سینیور

stability و strong skipping، فازها و کارایی، CompositionLocal



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۸. stability و strong skipping mode

از KOTLIN 2.0.20

strong skipping mode **به‌صورت پیش‌فرض روشن** است. کامپایلر Compose حالا همراه خود Kotlin منتشر می‌شود — با پلاگین `org.jetbrains.kotlin.plugin.compose` و هم‌نسخه با Kotlin.

	قبل از strong skipping	با strong skipping (پیش‌فرض)
composable با پارامتر unstable	هیچ‌وقت skip نمی‌شد	قابل skip
مقایسهٔ پارامترهای unstable	—	برابری نمونه (`===`)
مقایسهٔ پارامترهای stable	با `equals()`	با `equals()`
lambdaهای داخل composable	فقط اگر همهٔ captureها stable بودند remember می‌شدند	همیشه remember می‌شوند؛ captureها نقش key دارند

stable: تایپ‌های اولیه، `String`، تایپ‌های تابعی، enumها، کلاس‌هایی که همهٔ propertyهای عمومی‌شان `val` و از تایپ stable هستند، تایپ‌های `@Immutable` یا `@Stable`، و کالکشن‌های immutable در kotlinx.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۸. چه چیزی هنوز خراب می‌شود

```
// Unstable: a `var`, and List is an interface
data class CartUi(var items: List<Item>)

// Stable (if Item is stable): val + immutable collection
data class CartUi(val items: ImmutableList<Item>)
```

```
composeCompiler {
    reportsDestination = layout.buildDirectory.dir("compose")
    metricsDestination = layout.buildDirectory.dir("compose")
    stabilityConfigurationFiles.add(
        rootProject.layout.projectDirectory
            .file("stability.conf")
    )
}
```

تلهٔ مصاحبه

«*data classها stable هستند.*» — نسخهٔ پارسال همین را گفته بود. فقط وقتی stable است که همهٔ propertyها `val` و از تایپ stable باشند. یک `var`، یک `List` یا کلاسی از ماژولی **بدون** کامپایلر Compose آن را unstable می‌کند.

تلهٔ مصاحبه

«*strong skipping دیگر stability را بی‌اهمیت کرده.*» — پارامترهای unstable با `===` مقایسه می‌شوند. یک `List` جدید ولی با محتوای برابر (هر `map {}` در ViewModel) باز هم recomposition می‌دهد. و lambdaها دیگر به `remember` دستی نیاز ندارند — توصیهٔ نسخهٔ پارسال کهنه شده است.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۹. سه فاز — خواندن state را عقب بیندازید

```
Composition ──► Layout (measure, place) ──► Drawing
 reads here:     reads here:                  reads here:
 re-run all 3    re-run layout + draw         re-run draw
```

```
val color by animateColorAsState(target, label = "bg")
val fade by animateFloatAsState(targetAlpha, label = "a")

// Read in composition: recomposes on every animation frame
Box(Modifier.background(color))

// Read in the draw phase: no recomposition
Box(Modifier.drawBehind { drawRect(color) })

// Lambda versions defer the read to layout / draw
Box(Modifier.offset { IntOffset(0, scroll.value / 2) })
Box(Modifier.graphicsLayer { alpha = fade })
```

تلهٔ مصاحبه

«`graphicsLayer` هیچ‌وقت *recomposition ایجاد نمی‌کند.*» — فقط نسخهٔ **lambda** که state را داخل بلاک می‌خواند.

`Modifier.graphicsLayer(alpha = fade)` مقدار `fade` را در composition می‌خواند، پس در هر فریم recompose می‌شود.



اسماعیل مرادی

@esmaeeil-moradi700

سؤال ۹. چک‌لیست پایهٔ کارایی

- کارایی را روی **release** build با R8 و Baseline Profile بسنجید (روز ۱) — Compose در build دیباگ خیلی کندتر و گمراه‌کننده است
- **Layout Inspector** تعداد recomposition و skip هر composable را نشان می‌دهد
- **گزارش‌های کامپایلر Compose**: کدام composableها قابل skip هستند و کدام پارامترها unstable
- کار سنگین (مرتب‌سازی، فرمت‌کردن، parse) را از composition بیرون ببرید — به ViewModel یا `remember(keys)`
- jank واقعی را با system tracing و Perfetto پیدا کنید — بررسی عمیقش در روز ۱۷ است

تلهٔ مصاحبه

نوشتن رو به عقب (backwards write): نوشتن در یک state object بعد از خواندنش در همان composition — مثلاً `count++` مستقیم در بدنه. یک recomposition دیگر زمان‌بندی می‌شود که دوباره می‌نویسد: یک حلقهٔ بی‌پایان.

تلهٔ مصاحبه

«*برای اطمینان همه جا* `@Stable` *بگذارید.*» — این یک **قول** به کامپایلر است، نه یک بررسی. اگر دربارهٔ یک کلاس تغییرپذیر دروغ بگویید، Compose به‌روزرسانی‌ها را skip می‌کند: UI کهنه‌ای که دیباگش سخت است.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۰. CompositionLocal: مقدارهای ضمنی در محدودهٔ درخت

```
@Immutable
data class Spacing(
    val small: Dp = 4.dp,
    val medium: Dp = 8.dp,
    val large: Dp = 16.dp,
)

val LocalSpacing = staticCompositionLocalOf { Spacing() }

val roomy = Spacing(large = 24.dp)
CompositionLocalProvider(LocalSpacing provides roomy) {
    Content() // reads LocalSpacing.current.large
}
```

	`compositionLocalOf`	`staticCompositionLocalOf`
تغییر مقدار	فقط **خواننده‌ها** recompose می‌شوند	**کل** محتوای provideشده recompose می‌شود
مناسب برای	مقدارهایی که زیاد تغییر می‌کنند	مقدارهایی که به‌ندرت یا هیچ‌وقت تغییر نمی‌کنند (توکن‌های تم)



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۰. کِی از آن استفاده نکنیم

- کاربردهای درست: توکن‌های تم، `LocalContext`، `LocalDensity`، `LocalLayoutDirection` — در محدودهٔ درخت و با یک مقدار پیش‌فرض معقول
- هر خواندن یک **وابستگی ضمنی** است: در امضای تابع دیده نمی‌شود و در preview و تست به‌راحتی فراموش می‌شود

تلهٔ مصاحبه

استفاده از CompositionLocal برای رساندن ViewModel، repository یا دادهٔ صفحه «تا پارامتر کمتری داشته باشیم». وابستگی‌ها را پنهان می‌کند و preview را خراب می‌کند. پارامتر صریح پاس بدهید؛ برای سرویس‌ها از DI استفاده کنید (روز ۷).

تلهٔ مصاحبه

`LocalContext.current as Activity` — ممکن است context یک `ContextWrapper` باشد (در دیالوگ‌ها یا wrapperهای تم)، پس این cast می‌تواند crash کند. زنجیرهٔ `baseContext` را باز کنید، یا بهتر: در کد UI اصلاً به Activity نیاز نداشته باشید.



اسماعیل مرادی
@esmaeeil-moradi700

آرکیتکت

تم Material 3، API کامپوننت‌های design system، Compose در مقیاس بزرگ



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۱. تم در Material 3

```
@Composable
fun AppTheme(
    darkTheme: Boolean = isSystemInDarkTheme(),
    dynamicColor: Boolean = true,
    content: @Composable () -> Unit,
) {
    val context = LocalContext.current
    val colors = when {
        dynamicColor && Build.VERSION.SDK_INT >= 31 ->
            if (darkTheme) dynamicDarkColorScheme(context)
            else dynamicLightColorScheme(context)
        darkTheme -> DarkColors
        else -> LightColors
    }
    MaterialTheme(colorScheme = colors, typography = AppTypo) {
        CompositionLocalProvider(
            LocalSpacing provides Spacing(),
            content = content,
        )
    }
}
```

- کامپوننت‌ها از **نقش‌های رنگ** مثل `colorScheme.primary` و `onSurface`، مقیاس تایپوگرافی و shapeها استفاده می‌کنند — هرگز از کد hex
- رنگ پویا (dynamic color) از اندروید ۱۲ (API 31)؛ رنگ‌های برند به‌عنوان جایگزین
- توکن‌هایی که Material ندارد (فاصله‌گذاری، سطوح elevation) در CompositionLocalهای خودتان



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۱. تله‌های تم

تلهٔ مصاحبه

`Color(0xFF…)` هاردکد داخل کامپوننت‌ها. تم تیره، رنگ پویا و تم‌های کنتراست بالا بی‌صدا خراب می‌شوند. فقط فایل تم باید کدهای hex را بشناسد.

تلهٔ مصاحبه

قاطی کردن کامپوننت‌های Material 2 (یعنی `androidx.compose.material`) با Material 3. کامپوننت‌های M2 از `MaterialTheme` مربوط به **M2** می‌خوانند، پس داخل تم M3 به رنگ‌ها و تایپوگرافی پیش‌فرض برمی‌گردند.

تلهٔ مصاحبه

نادیده گرفتن window insetها. با targetSdk 35 و بالاتر، edge-to-edge اجباری است: محتوا زیر نوارهای سیستم رسم می‌شود. از `innerPadding` که `Scaffold` می‌دهد، یا از `Modifier.safeDrawingPadding()` استفاده کنید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۲. API کامپوننت‌های design system

```
@Composable
fun AppCard(
    title: @Composable () -> Unit,
    modifier: Modifier = Modifier,
    actions: @Composable RowScope.() -> Unit = {},
    content: @Composable ColumnScope.() -> Unit,
) {
    val spacing = LocalSpacing.current
    Card(modifier) {
        Column(Modifier.padding(spacing.large)) {
            val style = MaterialTheme.typography.titleMedium
            ProvideTextStyle(style) { title() }
            content()
            Row(Modifier.align(Alignment.End)) { actions() }
        }
    }
}
```

- ترتیب پارامترها طبق راهنمای API در Compose: اول پارامترهای الزامی، بعد `modifier`، بعد اختیاری‌ها و در آخر `content`
- **slot**ها (lambdaهای `@Composable`) به‌جای رشته و flag: صدازننده تصمیم می‌گیرد داخلش چه باشد
- پیش‌فرض بدون state؛ preview برای هر حالت؛ screenshot test در CI (روز ۱۰)



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۲. اداره کردن یک کتابخانهٔ کامپوننت

تلهٔ مصاحبه

انفجار booleanها: یعنی `AppButton(isPrimary, isOutlined, hasIcon, isLoading, …)`. هر flag تعداد حالت‌های قابل‌تست را دو برابر می‌کند. آن را به `PrimaryButton` و `OutlinedButton` بشکنید و برای آیکون از slot استفاده کنید.

تلهٔ مصاحبه

«*design system مشترک بین اندروید و iOS برای هر کامپوننت به `expect` و `actual` نیاز دارد.*» — توصیهٔ نسخهٔ پارسال. با **Compose Multiplatform** کامپوننت‌ها در `commonMain` زندگی می‌کنند؛ `expect` و `actual` فقط برای جزئیات نادر پلتفرم.

حاکمیت

قواعد سفارشی Lint (یا `compose-lints` از Slack) که استفادهٔ مستقیم از `material3.Button` را در ماژول‌های فیچر ممنوع می‌کنند، تیم‌ها را روی design system نگه می‌دارند. کتابخانه را با semver نسخه‌گذاری کنید و یادداشت مهاجرت منتشر کنید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۳. Compose در مقیاس بزرگ: interop

```
@Composable
fun LegacyChart(
    points: List<Point>,
    modifier: Modifier = Modifier,
) {
    AndroidView(
        factory = { context -> ChartView(context) },
        modifier = modifier,
        update = { view -> view.setPoints(points) },
        onRelease = { view -> view.clear() },
    )
}
```

- `factory` یک بار اجرا می‌شود؛ `update` هر بار که stateی که می‌خواند تغییر کند دوباره اجرا می‌شود
- در لیست‌های lazy، پارامتر `onReset` را بدهید تا View به‌جای ساخت دوباره **بازاستفاده** شود
- Compose داخل Fragment (با `ComposeView` و `ViewCompositionStrategy`) و Fragment داخل Compose (با `AndroidFragment`) در روز ۲ بررسی شد
- مهاجرت از پایین به بالا: اول تم، بعد کامپوننت‌های برگ، بعد صفحه‌های کامل

تلهٔ مصاحبه

رفتار با `update` مثل یک راه‌اندازی یک‌باره. هر بار که stateی که می‌خواند تغییر کند دوباره اجرا می‌شود — ارزان و idempotent نگهش دارید؛ listenerها را در `factory` بسازید، نه در `update`.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۳. محافظ‌های کارایی در CI

محافظ	چه چیزی را می‌گیرد
مقایسهٔ گزارش‌ها و metricهای کامپایلر Compose در CI	پارامترهای unstable جدید، composableهای غیرقابل skip
فایل پیکربندی stability	تایپ‌های domain از ماژول‌های بدون Compose که unstable حساب می‌شوند
Baseline Profile (روز ۱ و روز ۱۸)	فریم‌ها و اسکرول‌های اول کند
Macrobenchmark با `FrameTimingMetric`	پسرفت jank در اسکرول و انیمیشن
Lint و `compose-lints`	نبودن پارامتر `modifier`، state بدون remember، استفادهٔ غلط از effectها

جواب سینیور در یک جمله

«Compose هر چیزی را که state تغییرکرده را می‌خواند دوباره اجرا می‌کند. state را کوچک و بالا نگه دار، تایپ‌ها را stable، خواندن‌ها را دیرهنگام، و side effectها را داخل APIهای effect — بعد روی build ریلیز اندازه بگیر.»



اسماعیل مرادی
@esmaeeil-moradi700

روز ۸ — خلاصهٔ یک‌صفحه‌ای

- UI تابعی از state است. recomposition فقط scopeهایی را اجرا می‌کند که state تغییرکرده را **می‌خوانند**.
- بدنهٔ composable: سریع، idempotent و بدون side effect.
- `remember` از recomposition عبور می‌کند؛ `rememberSaveable` از چرخش صفحه و process death هم؛ ViewModel فقط از چرخش.
- state را بالا ببرید: مقدار به پایین، event به بالا. `modifier` اولین پارامتر اختیاری است.
- effectها: `LaunchedEffect` با key درست، `DisposableEffect` با `onDispose`، و `rememberCoroutineScope` برای callbackها (خودکار لغو می‌شود).
- ترتیب modifier یعنی معنا؛ modifier سفارشی جدید با `Modifier.Node`.
- لیست lazy: `key` یکتا، `contentType`، بدون اسکرول هم‌جهت تودرتو.
- strong skipping از Kotlin 2.0.20 پیش‌فرض است: پارامتر unstable با `===` مقایسه می‌شود؛ data class خودبه‌خود stable نیست.
- خواندن state را با modifierهای lambdaیی عقب بیندازید؛ روی build ریلیز اندازه بگیرید.
- تم با نقش‌های رنگ M3 و توکن‌های CompositionLocal — بدون کد hex در کامپوننت‌ها.

روز ۹ در راه است: تست، بخش اول. این پست را ذخیره کنید و همراه ما باشید.



اسماعیل مرادی
@esmaeeil-moradi700